

Funkcje zagnieżdzone, domknięcia (closures), dekoratory

Funkcje zagnieżdżone

Funkcje zagnieżdżone:

- funkcje zdefiniowane wewnątrz innych funkcji, tzw. opakowujących (enclosing functions),
- domyślnie mają dostęp do zmiennych funkcji opakowujących w trybie odczytu,
- możliwe jest wykorzystanie tzw. domykania zmiennych (closure), co jest istotne w dekorowaniu funkcji.

Funkcje zagnieżdżone

Przykład 1 :

```
def fun(message = 'hello'):      #funkcja opakowująca
    "This is the enclosing function"
    def inner_fun():            #funkcja zagnieżdżona
        "This is the nested function"
        print(message)          #zmienna zewnętrzna - message tylko do odczytu
    inner_fun()
    print(message)

fun()
```

```
# output:
hello
hello
```

Funkcje zagnieżdżone

Przykład 2 – zasięg zmiennych:

```
def fun(message = 'hello'):  
    "This is the enclosing function"  
    def inner_fun():  
        "This is the nested function"  
        message='goodbye' #utworzenie nowej lokalnej zmiennej message  
        print(message)  
    inner_fun()  
    print(message) #zmienna zewnętrzna message niezmienniona  
  
fun()
```

```
#output:  
goodbye  
hello
```

Funkcje zagnieżdżone

Przykład 3 - zasięg zmiennych:

```
def fun(message = 'hello'):
    "This is the enclosing function"
    def inner_fun():
        "This is nested function"
        nonlocal message    #utworzenie/uzyskanie dostępu do zmiennej zewnętrznej
        message
        message='goodbye'   #modyfikacja zmiennej zewnętrznej message
        print(message)
    inner_fun()
    print(message) #zmienna zewnętrzna message zmieniona

fun()
```

```
#output:
goodbye
goodbye
```

Funkcje zagnieżdżone

Przykład 4 - zasięg zmiennych:

```
def fun(message = 'hello'):  
    "This is the enclosing function"  
    def inner_fun():  
        "This is the nested function"  
        global message      #ustanowienie zmiennej globalnej message  
        message='goodbye'   #modyfikacja zmiennej globalnej message  
        print(message)  
    inner_fun()  
    print(message) #zmienna zewnętrzna message niezmienniona  
  
fun()  
print(message)
```

```
#output:  
goodbye  
hello  
goodbye
```

Funkcje zagnieżdżone

Przykład 5 – domknięcie zmiennej (closure):

```
def fun(message = 'hello'):
    "This is the enclosing function"
    def inner_fun():
        "This is nested function"
        return(message) #zapamiętanie zmiennej zewnętrznej message w inner_fun(), tzw. closure
    return inner_fun

new_fun1=fun('hello')    #domknięcie zmiennej message='hello'
print(new_fun1())

new_fun2=fun('goodbye')  #domknięcie zmiennej message='goodbye'
print(new_fun2())
```

```
#output:
hello
goodbye
```

Funkcje zagnieżdżone

Przykład 6 – domknięcie zmiennej (closure):

```
def fun(x):  
    def multiply(y):  
        return(x*y)  
    return multiply
```

```
multiply_by_2 = fun(2)  
print(multiply_by_2(9))
```

```
multiply_by_5 = fun(5)  
print(multiply_by_5(9))
```

#output:

18

45

Dekoratory

Dekoratory:

- konstrukcje umożliwiające dodawanie nowych operacji wykonywanych przez funkcje, metody, klasy,
- jest to tzw. metaprogramowanie – część kodu zmienia sposób działania innej części kodu.

Dekoratory

Przykład 7 – dekorowanie funkcji - klasyczne wywołanie dekoratora:

```
def fun(x):                #definicja funkcji
    print(x)

fun('Hello')               #działanie przed dekoracją

def dekoruj(old_fun):      #definicja funkcji dekorującej (dekorator)
    def new_fun(x):
        old_fun(x + ', man!')    #modyfikacja działania funkcji
    return new_fun

fun = dekoruj(fun)         #dekorowanie funkcji
fun('Hello')               #działanie po dekoracji
```

```
#output:
Hello
Hello, man!
```

Dekoratory

Przykład 8 – dekorowanie funkcji - wywołanie dekoratora z symbolem @:

```
def dekoruj(old_fun):          #funkcja dekoratora
    def new_fun(x):
        old_fun(x + ', man!')
    return new_fun

@dekoruj                      #dekorowanie funkcji
def fun(x):                   #definicja dekorowanej funkcji
    print(x)

fun('Hello')
```

```
#output:
Hello, man!
```

Dekoratory

Przykład 9 – dekorowanie funkcji – zmienna liczba argumentów:

```
def fun(*args,**kwds):    #funkcja przed dekoracją
    print(args)
    print(kwds)

fun("a","b","c",x=6,z=8)    #wywołanie funkcji

def dekoruj(old_fun):    #dekorator
    def new_fun(*args, **kwargs): #po dekoracji funkcja dodatkowo zlicza parametry
        old_fun(*args, **kwargs)
        print('Num of parameters: ', len(args) + len(kwargs))
    return new_fun

fun=dekoruj(fun)    #dekorowanie
print('---po dekoracji---')
fun("a","b","c",x=6,y=7,z=8)    #działanie po dekoracji
fun("b","c",x=6,y=7,z=8) )
```

```
#output:
('a', 'b', 'c')
{'x': 6, 'z': 8}
---po dekoracji---
('a', 'b', 'c')
{'x': 6, 'y': 7, 'z': 8}
Num of parameters: 6
('b', 'c')
{'x': 6, 'y': 7, 'z': 8}
Num of parameters: 5
```

Dekoratory

Przykład 10 – modyfikacja wyjścia:

```
def double_out(old_fun):  
    def new_fun(x,y):  
        return 2*old_fun(x,y)    #modyfikacja wyjścia  
    return new_fun  
  
@double_out  
def fun(x,y):  
    return x+y  
  
print(fun(2,2))
```

#output:

8

Dekoratory

Przykład 11 – modyfikacja wejścia:

```
def double_in(old_fun):  
    def new_fun(x,y):  
        x=x+1          #modyfikacja wejścia  
        y=y+1  
        return old_fun(x,y)  
    return new_fun  
  
@double_in  
def fun(x,y):  
    return x+y  
  
print(fun(2,2))
```

#output:

6

Dekoratory

Przykład 12 – dodawanie sprawdzania argumentu:

```
def check_in(old_fun):  
    def new_fun(x,y):  
        if y==0:  
            return "Niedozwolona wartość argumentu!"  
        return old_fun(x,y)  
    return new_fun
```

```
@check_in  
def fun(x,y):  
    return x/y
```

```
print(fun(6,2))  
print(fun(2,0))
```

#output:

3.0

Niedozwolona wartość argumentu!

Dekoratory

Przykład 13 – ,fabryka' dekoratorów (modyfikacja wyjścia – zmiana mnożnika):

```
def gen_dec(multiplier): #fabryka dekoratorów – funkcja, która zwraca dekorator
    def dekorator(old_fun): #definicja dekoratora
        def new_fun(x):
            return multiplier * old_fun(x)
        return new_fun
    return dekorator
```

```
@gen_dec(3) #dekorator 1
def fun(num):
    return num
print(fun(3))
```

```
@gen_dec(5) #dekorator 2
def fun(num):
    return num
print(fun(3))
```

```
#output:
9
15
```

Dekoratory

Przykład 14 – ,fabryka’ dekoratorów (modyfikacja wejścia – sprawdzanie typu):

```
def gen_dec(correct_type):
    def type_factory(old_fun):
        def new_fun(arg):
            if (type(arg) == correct_type):
                return old_fun(arg)
            else:
                return 'Bad type!'
        return new_fun
    return type_factory
```

```
@gen_dec(int)
def fun(x):
    return x*2
print(fun(2))
print(fun('Hello'))
```

```
@gen_dec(str)
def fun(x):
    return len(x)
print(fun(' Hello'))
print(fun(2))
```

#output:

4

Bad type!

6

Bad type!

Dekoratory

Przykład 15 – kaskada dekoratorów:

```
def double_out(old_fun): #dekorator - modyfikacja wyjścia
    def new_fun(x):
        return 2*old_fun(x)
    return new_fun

def check_input(old_fun): #dekorator - sprawdzanie typu
    def new_fun(x):
        if not isinstance(x,int):
            return "Niedozwolony typ!"
        return old_fun(x)
    return new_fun

@check_input
@double_out
def fun(x):
    return x

#fun=check_input(double_out(fun)) - alternatywne wywołanie dekoratorów

print(fun(2))
print(fun('asd'))
```

#output:
4
Niedozwolony typ!

Zadanie 1

Zakładamy, że będziemy definiować funkcje dwuargumentowe (sumowanie, mnożenie, itp.), które zwracają wartość całkowitą (integer).

Napisz dekorator, który zamienia wyjście w/w funkcji (int) na string zawierający nazwy cyfr np. wyjście funkcji to 35, a po udekorowaniu to ,trzy pięć’.

Przetestuj działanie na kilku funkcjach.

Zadanie 2

Zmodyfikuj zadanie 1 tak, aby dekorator dodatkowo sprawdzał, czy wyjście z funkcji dekorowanej jest liczbą całkowitą. Jeśli nie to powinien być generowany wyjątek „NotInteger”.

Przetestuj działanie wyjątku.

Zadanie 3

Zakładamy, że będziemy definiować funkcje o zmiennej liczbie argumentów, a które zwracają wartość całkowitą (integer) będącą sumą argumentów.

Napisz dekorator, który przepuszcza do dekorowanej funkcji tylko argumenty typu całkowitego.

Przetestuj działanie na kilku funkcjach.

Zadanie 4

Napisz dekorator, który będzie zliczał wywołania wszystkich dekorowanych funkcji.

Przetestuj działanie na kilku funkcjach. Spróbuj je wywołać w pętli.

Zadanie 5

Zakładamy, że mamy definicję funkcji (np. `get_temp(t)`), która pobiera argument - temperaturę w kelwinach i zwraca temperaturę także w kelwinach.

Napisz funkcję - ,fabrykę' dekoratorów, która będzie przyjmowała parametr w postaci ciągu: ,celcjusz', ,fahrenheit', itp. i zwróci odpowiedni dekorator, który będzie odpowiednio modyfikował wyjście funkcji `get_temp()`, aby zwracała temperaturę w odpowiedniej skali.

Przetestuj działanie.